Intermediate (chunking method changes the answer) — Solution

Back to the exercise · Hint 1 · Hint 2 · Solution

Story — `chunking_practice.py`: the same document, split two different ways, can hand the model two different answers to the same question — this is the exercise that makes "chunking is a real design decision" something you watched happen, not something you took on faith. **If not:** the Build Task's chunker would be the first place a split fact ever bit you, in a pipeline with far more moving parts to debug through.

All examples below use the same test document and question — put them at the top of your practice file:

```
# chunking_practice.py
PARAGRAPH_1 = (
    "Coffee brewing starts with the grind size. A coarse grind works best "
    "for a French press, because its metal filter lets fine grounds slip "
    "through and makes the coffee taste muddy and bitter. A fine grind is "
    "right for espresso instead, where water touches the grounds for only "
    "a few seconds."
)
PARAGRAPH_2 = (
    "Water temperature matters just as much as grind size. Water that is "
    "too hot, above 205 degrees Fahrenheit, scalds the coffee and pulls "
    "out bitter compounds that should have stayed in the grounds. Water "
    "that is too cool under-extracts the coffee, leaving it weak and sour "
    "even with plenty of grounds."
)
PARAGRAPH_3 = (
    "The final step is the pour itself. A slow, steady pour in a spiral "
    "pattern wets all the grounds evenly, which is why baristas practice "
    "just the pour for weeks before worrying about anything else."
)
# a blank line ("\n\n") between paragraphs, like a real text file
DOCUMENT = PARAGRAPH_1 + "\n\n" + PARAGRAPH_2 + "\n\n" + PARAGRAPH_3

QUESTION = "What water temperature should be avoided when brewing coffee?"
```

Solution

```
# chunking_practice.py
import math
from openai import OpenAI

client = OpenAI()

# DOCUMENT and QUESTION: the block at the top of this page

def chunk_by_chars(text: str, chunk_size: int = 200) -> list[str]:
    chunks = []
    for i in range(0, len(text), chunk_size):
        chunks.append(text[i:i + chunk_size])
    return chunks

def chunk_by_paragraph(text: str) -> list[str]:
    chunks = []
    for piece in text.split("\n\n"):
        stripped = piece.strip()
        if stripped:
            chunks.append(stripped)
    return chunks

def get_embedding(text: str) -> list[float]:
    response = client.embeddings.create(
        model="text-embedding-3-small", input=text
    )
    return response.data[0].embedding

def cosine_similarity(a: list[float], b: list[float]) -> float:
    dot = 0.0
    for x, y in zip(a, b):
        dot = dot + x * y
    squares = 0.0
    for x in a:
        squares = squares + x * x
    norm_a = math.sqrt(squares)
    squares = 0.0
    for x in b:
        squares = squares + x * x
    norm_b = math.sqrt(squares)
    return dot / (norm_a * norm_b)

def find_best_chunk(
    chunks: list[str], question_vector: list[float],
```

```
) -> tuple[str, float]:
    best_chunk = ""
    best_score = -1.0
    for chunk in chunks:
        chunk_vector = get_embedding(chunk)
        score = cosine_similarity(question_vector, chunk_vector)
        if score > best_score:
            best_score = score
            best_chunk = chunk
    return best_chunk, best_score

# Compare two chunking methods: which gives the best-matching chunk?
question_vector = get_embedding(QUESTION)
methods = {
    "chunk_by_chars": chunk_by_chars(DOCUMENT),
    "chunk_by_paragraph": chunk_by_paragraph(DOCUMENT),
}
for name, chunks in methods.items():
    best_chunk, best_score = find_best_chunk(chunks, question_vector)
    print(f"{name} best match, score: {best_score:.4f}")
    print(best_chunk)
    print()
```

Expected output: same conclusion as Basic — `chunk_by_paragraph`'s top match contains the whole temperature fact; `chunk_by_chars`'s top match is missing half of "scalds."